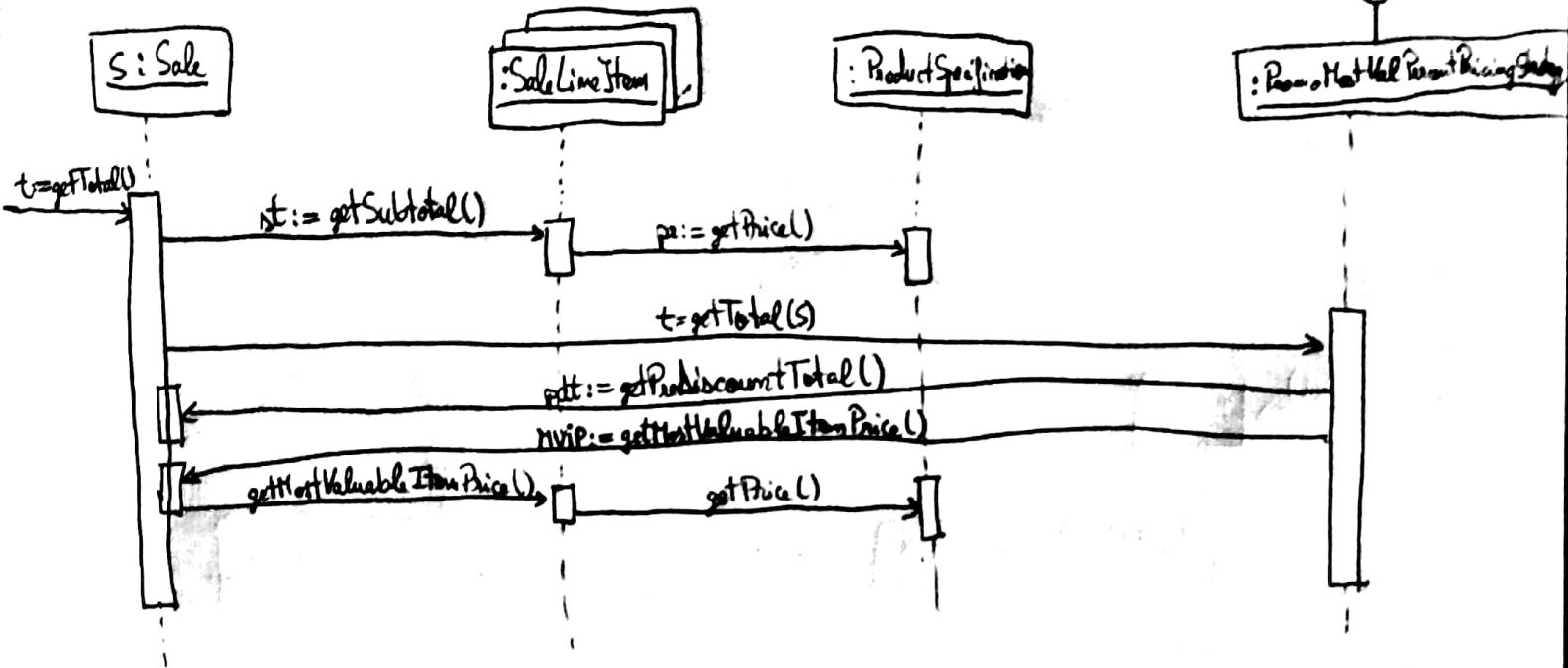


S3

POTRA
BOGDAN
GR 20442

ISalePricingStrategy



The "Sale" knows how to compute the total amount to be paid by summing up all sub-totals. $\Rightarrow$ Information Expert (IE)

The "SaleLineItem" knows how to compute the subtotal by quantity & price. $\Rightarrow$ IE

The "ProductSpecification" has to know its own price. $\Rightarrow$ IE

The Strategy pattern was used together with polymorphism and protected variations in order to be able to define several pricing strategies based on current/future promotions.

By using the strategy, the final price to be paid is $t = pdt - mvip \cdot \frac{10}{100}$ if the pdt is greater than 200 LEI. The pdt (pre-discount-total) is previously calculated by the "Sale", by IE. Furthermore, $mvip$ (most-valuable-item-price) is also obtained by the "Sale" by querying all the "SaleLineItem"s, again by IE.

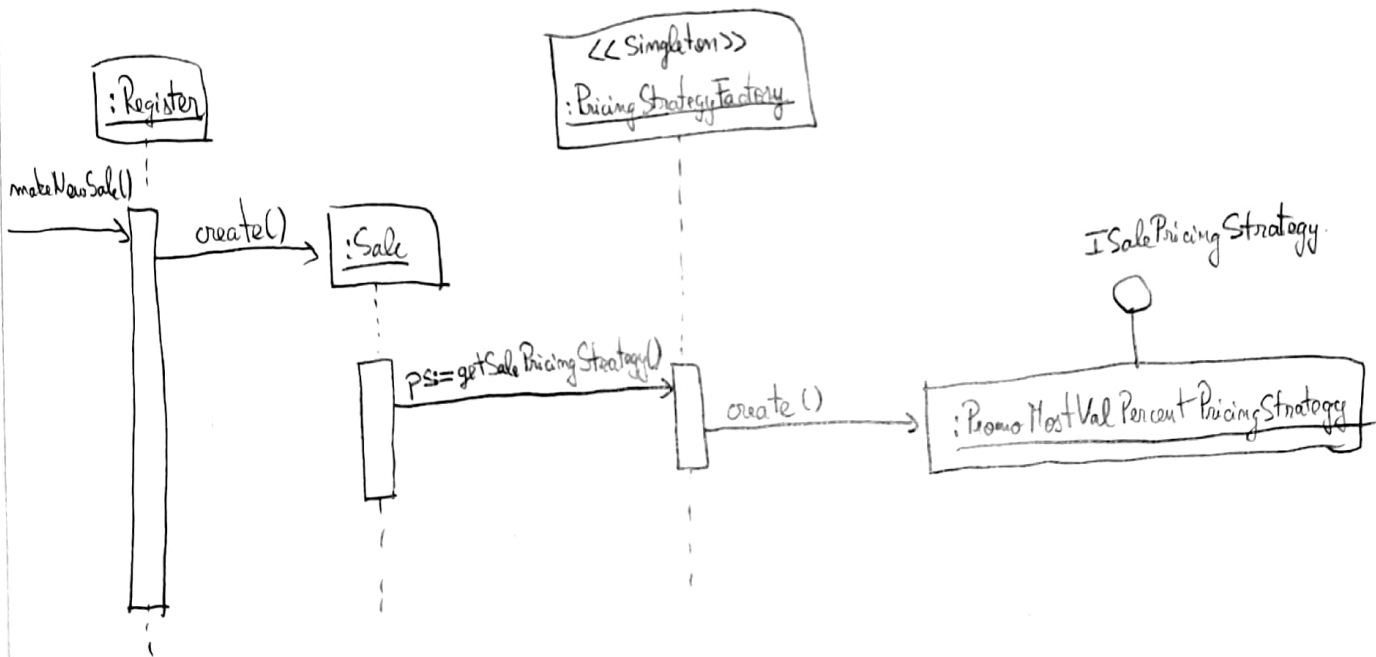
This design does not introduce new dependencies $\Rightarrow$ low coupling.

This design does not add unrelated responsibilities $\Rightarrow$ high cohesion.

S3 - continued

POTRA
BOGDAN
GR. 30442

Of course, the pricing strategy should be created somehow. For this, I propose the use of a pure fabrication, singleton Factory class.



The Singleton, Factory ensure the creation of only 1 such strategy per system.
As stated before, the "ISalePricingStrategy" interface provides polymorphism and protected variations.
The design supports low coupling and high cohesion.